lock on the tail mutex. Because you need to access `tail` only once, you can just acquire the mutex for the time it takes to do the read. This is best done by wrapping it in a function. In fact, because the code that needs the `head` mutex locked is only a subset of the member, it's clearer to wrap that in a function too. The final code is shown here.

Listing 6.6. A thread-safe queue with fine-grained locking

```
template<typename T>
class threadsafe_queue
{
private:
    struct node
     {
         std::shared_ptr<T> data;
         std::unique_ptr<node> next;
     };

     std::mutex head_mutex;
     std::unique_ptr<node> head;
     std::mutex tail_mutex;
     node* tail;

     node* get_tail()
     {
         std::lock_guard<std::mutex> tail_lock(tail_mutex);
         return tail;
     }

     std::unique_ptr<node> pop_head()
     {
         std::lock_guard<std::mutex> head_lock(head_mutex);

         if(head.get()==get_tail())
         {
             return nullptr;
         }
         std::unique_ptr<node> old_head=std::move(head);
         head=std::move(old_head->next);
         return old_head;
    }

public:
    threadsafe_queue():
         head(new node),tail(head.get())
     {}

     threadsafe_queue(const threadsafe_queue& other)=delete;
```